

# Relatório de Implementação — Padrões de Teste (Test Patterns)

**Disciplina:** Teste de Software

**Professor:** Cleiton Tavares

**Instituição:** PUC Minas

---

## 1. Padrões de Criação de Dados (Builders)

### 1.1 Por que CarrinhoBuilder e não CarrinhoMother?

O Carrinho é um objeto composto: ele carrega um `User` e uma lista de `Item`. Os cenários de teste exigem variações arbitrárias — carrinho vazio, com um item, com múltiplos itens, para usuário padrão ou premium. Um Object Mother para `Carrinho` resultaria em explosão combinatória de métodos estáticos:

```
// Object Mother - inviável para objeto composto  
CarrinhoMother.carrinhoVazio()  
CarrinhoMother.carrinhoComUmItem()  
CarrinhoMother.carrinhoComDoisItens()  
CarrinhoMother.carrinhoPremiumComDoisItens()  
// ... infinitas combinações
```

O Data Builder resolve isso com uma API fluente que declara **apenas o que é relevante** para cada teste:

```
const carrinho = new CarrinhoBuilder()  
  .comUser(UserMother.umUsuarioPremium())  
  .comItens([ { nome: 'Produto A', preco: 100 } ])  
  .build();
```

## 1.2 UserMother — Object Mother para entidades simples

Já o `User` tem variações semânticas fixas (`PADRAO` ou `PREMIUM`), sem combinações arbitrárias. Aqui o Object Mother é a escolha certa:

```
const usuario = UserMother.umUsuarioPremium();  
// Intenção clara, sem configuração
```

Se usássemos um Builder para `User`, teríamos uma API fluente onde cada método `.comTipo()` seria chamado apenas uma vez por teste — complexidade desnecessária.

## 1.3 Legibilidade e Manutenção

- Mudanças no construtor de `Carrinho` afetam apenas o Builder, não cada teste individual
  - O Builder declara **intenção** (`.comUser()`, `.comItens()`); o setup manual declara **implementação** (`new Carrinho()`, `.adicionarItem()`)
  - Novos cenários são adicionados sem duplicar código de setup
- 

## 2. Padrões de Test Doubles (Mocks vs. Stubs)

### 2.1 Cenário: Falha no Pagamento (Stub)

No teste de falha, o `GatewayPagamento` é um **Stub**: configuramos para retornar `{ success: false }` e verificamos o **estado** do sistema (o pedido retornado é `null`). Não nos importa quantas vezes foi chamado ou com quais argumentos — só precisamos controlar o fluxo.

```
const gatewayStub = {  
  cobrar: jest.fn().mockResolvedValue({ success: false })  
};  
  
// Assert - verificação de estado  
expect(pedido).toBeNull();
```

As dependências que **não deveriam** ser chamadas nesse fluxo (`PedidoRepository`, `EmailService`) são **Dummies** — objetos que existem apenas para preencher a interface,

com asserções extras garantindo que não foram invocados:

```
expect(repositoryDummy.salvar).not.toHaveBeenCalled();  
expect(emailDummy.enviarEmail).not.toHaveBeenCalled();
```

## 2.2 Cenário: Sucesso Premium (Stub + Mock)

No teste de sucesso premium, usamos **três tipos de doubles**:

| Dependência      | Tipo | Justificativa                                                                                                                                                                      |
|------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GatewayPagamento | Stub | Controla o fluxo ( <b>success: true</b> ). A verificação de <code>cobrar(180, ...)</code> valida a <b>correção do fluxo principal</b> (desconto aplicado), não um efeito colateral |
| PedidoRepository | Stub | Retorna um pedido simulado. Não verificamos interação — só precisamos que exista                                                                                                   |
| EmailService     | Mock | O envio de e-mail é um <b>efeito colateral observável</b> . Verificamos: foi chamado? Quantas vezes? Com quais argumentos?                                                         |

```
// Mock - verificação de comportamento  
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);  
expect(emailMock.enviarEmail).toHaveBeenCalledWith(  
    'premium@email.com',  
    'Seu Pedido foi Aprovado!',  
    expect.any(String)
```

);

## 2.3 Verificação de Estado vs. Verificação de Comportamento

- **Verificação de Estado (Stub):** “Dado que o pagamento falhou, o resultado é `null`?”  
— testamos o **output** do SUT
- **Verificação de Comportamento (Mock):** “O sistema se comportou corretamente enviando o e-mail?” — testamos a **interação** com a dependência

Como define Martin Fowler: “*Mocks são objetos pré-programados com expectativas que formam uma especificação das chamadas que devem receber.*”

---

## 3. Conclusão

A aplicação combinada dos padrões produziu testes que são:

- **Legíveis:** o setup declarativo (Builder + Object Mother) comunica intenção, não implementação
- **Determinísticos:** Stubs eliminam dependências externas (rede, banco), removendo não-determinismo
- **Rápidos:** sem chamadas reais a gateway ou banco, os testes executam em milissegundos
- **Mantíveis:** mudanças nos construtores das entidades afetam apenas os builders, não cada teste

Os antipadrões evitados:

---

| Test Smell           | Como evitamos                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------|
| <i>Obscure Setup</i> | Builders com API fluente tornam o setup explícito                                                        |
| <i>Mystery Guest</i> | Dummies com <code>not.toHaveBeenCalled()</code> garantem que dependências não são acionadas sem intenção |

| Test Smell                | Como evitamos                                           |
|---------------------------|---------------------------------------------------------|
| <i>Assertion Roulette</i> | Cada teste tem asserções focadas em um único cenário    |
| <i>Fragile Test</i>       | Stubs isolam o SUT de mudanças em dependências externas |